

Technical Design Document

Università degli Studi di Milano

Dipartimento di Informatica



Andrea Tettamanti 84319A

Lorenzo Vernavà 83218A

Domenico Coricciati 73950A

**BOUND
BY LIGHT**

April 17, 2026

Contents

1	Project Goal	2
2	Provided Services	4
3	Client Side	5
3.1	Hardware Requirements	5
3.2	Software Requirements	5
4	Workload Estimation	7
5	General Architecture	13
5.1	Frontend	14
5.2	Backend	16
5.3	Workload Capacity	20
5.4	Network Synchronization and State Management	24
5.5	Production Migration Path: from Open Beta to Commercial Release . . .	27
6	Quality Assurance and Testing Strategy	32
7	Physical Services	36
8	Delivery	38
8.1	Delivery Platforms	38
8.2	Estimated Delivery Time	38
8.3	Estimated Production Workflow	38
9	Cost Estimation and Financial Strategy	40
9.1	Human Resources and Compensation	40
9.2	Technical Infrastructure and Friend Pass Impact	40
9.3	Consolidated Budget Table	40
9.4	Business Model: Friend Pass and Break-even Analysis	41

1 Project Goal

Bound by Light (BBL) is an online-mandatory two-player cooperative shooter for PC (Windows). From a technology standpoint, the goal of this document is to describe the infrastructure and software architecture required to run the game as an **online service**, not merely as a distributed application, for the duration of the open beta and beyond.

A modern online game is no longer a piece of software that is distributed and played locally: it is a service that is run continuously, and the quality of that service is an inseparable part of the product. *Bound by Light* is explicitly designed around this principle. Online connectivity is not a feature that can be disabled; it is the core component around which every mechanic, every session structure, and every save-management decision has been made (see GDD §8.9.8). Shutting down the server would make the game unplayable.

The concrete technology objectives are the following.

24/7 availability for the open beta. The infrastructure must sustain continuous availability across the beta period. Sessions are self-contained and short (30–60 minutes as per GDD §8.4.12), so the system must support players connecting and disconnecting at any time without impacting other active sessions.

Real-time two-player co-op over the Internet. The game requires two clients to share a continuous, low-latency, coherent game state. The tether mechanic, shared Bond Energy bar, simultaneous combat, and information-asymmetry design (GDD §8.9.7) all impose hard requirements on synchronisation: both players must observe the same world state within a bounded delay. Given that the target audience is casual to midcore players using broadband home connections (GDD §3.3), a round-trip latency target of ≤ 100 ms end-to-end is considered acceptable. It is well established in the online game networking literature that for real-time games, bound transmission delay, not raw bandwidth, is the critical QoS parameter.

Steam-native experience. Distribution is through Steam as the primary channel (GDD §3.2). The online infrastructure must leverage Valve’s existing platform services: Steam authentication, the Friend Pass system, invitations, and the Steam Datagram Relay (SDR) network for transport. This offloads identity management, NAT traversal, and relay routing from the team while providing a seamless in-client experience to players. The game does not implement its own authentication backend; Steam IDs are the canonical player identifiers.

Persistent progression and session-code management. *Bound by Light* introduces a session-code mechanism that permanently links two Steam accounts to a single save file (GDD §8.9.8). Permanent upgrades persist across sessions and must not be lost on disconnection. A custom cloud-hosted backend is required to manage this persistence reliably and independently of client state.

Scalability from beta to launch. The open beta targets a limited concurrent user base (estimated in §5.3). The architecture must be designed so that capacity can be extended incrementally as user numbers grow, without requiring a full infrastructure redesign.

Minimal operational overhead for an indie team. Juggernaut is a small development team. The infrastructure must prioritise managed cloud services and platform-native tools over self-hosted solutions wherever possible, in order to reduce operational burden and keep costs proportionate to the scale of the beta.

2 Provided Services

The services the team will provide include the management of user-reported bugs through an integrated logging system for crashes and dedicated game forums. Official Reddit forums will be established, and the project will be promoted via YouTube.

Additionally, the development team, or an external social media manager, will maintain an Instagram page to publish key news regarding updates, gameplay, and trailers.

An official Discord server will also be created to foster closer interaction with users. This server features voice channels for community gathering, Q&A text channels, dedicated announcement channels, and general chat areas for users.

3 Client Side

This chapter defines the requirements that must be met by the end-user’s machine in order to run *Bound by Light*. Requirements are divided into hardware specifications (§3.1) and software dependencies (§3.2).

3.1 Hardware Requirements

The hardware requirements for the player’s PC are defined in GDD §3.3 and are reproduced here for completeness. The host player’s machine bears a higher computational load than the guest, since it runs both the authoritative game simulation and the local rendering pipeline simultaneously (see §5.2).

Component	Minimum	Recommended
CPU	Intel Core i5-6600 / AMD Ryzen 5 1600	Intel Core i7-9700K / AMD Ryzen 7 3700X
RAM	8 GB	16 GB
GPU	NVIDIA GTX 1050 / AMD RX 570 (4 GB VRAM)	NVIDIA GTX 1660 / AMD RX 5600 XT (6 GB VRAM)
Storage	8 GB available	10 GB SSD
Network	Broadband internet connection	Broadband internet connection
DirectX	Version 11	Version 12

Table 2: Client hardware requirements for *Bound by Light* (from GDD §3.3).

3.2 Software Requirements

The following software must be present on the player’s machine. With the exception of the Steam client and the operating system, all runtime dependencies are bundled with the game installation and require no separate action from the player.

Operating System. Windows 10 (64-bit, version 1903 or later) or Windows 11 (64-bit). No macOS or Linux build is planned for the beta.

Steam Client (mandatory). The game is distributed exclusively through Steam. The Steam client must be installed and the player must be logged into an active Steam account before launching the game. The Steam client handles download, update delivery, local authentication, and overlay integration. This requirement is a direct consequence of the Steam-native architecture described in §5.1.

DirectX Runtime. DirectX 11 (minimum) or DirectX 12 (recommended). The appropriate version is installed as part of standard Windows updates; no separate installation is required in most configurations.

FMOD Runtime (bundled). *Bound by Light* uses FMOD as its spatial and procedural audio engine. The FMOD runtime libraries are statically linked into the game build and do not require a separate installation by the player.

Visual C++ Redistributables (bundled). The Unity build pipeline generates a Windows executable that depends on Microsoft Visual C++ Redistributable packages. These are included in the game installer and installed automatically if not already present on the system.

4 Workload Estimation

This chapter provides a structured, bottom-up estimate of the infrastructure load that *Bound by Light* is expected to generate during the open beta period. The goal is to derive quantitative dimensioning targets that the architecture described in §5 is designed to satisfy. The analysis is divided into four stages: user base projection, per-session resource profiling, aggregate load computation, and cost projection.

Scope and Assumptions

The estimates in this chapter apply to the **open beta phase**. The following assumptions are adopted throughout:

- The beta is targeted at a **Steam-only PC audience**. No console or cross-platform traffic is considered.
- Sessions are **two-player only**: every active game session corresponds to exactly two concurrently connected players.
- The beta is launched in a **controlled ramp-up**: not all registered users will be online simultaneously. Peak concurrency is estimated based on observed patterns from comparable indie co-op titles on Steam.
- Infrastructure is **not responsible for real-time game traffic**: all NGO state-replication packets are routed through Steam’s Datagram Relay (SDR) and do not burden our own servers (see §5).

User Base Projection

Reference titles. *Bound by Light* targets the same market segment as two-player cooperative PC titles distributed on Steam. Representative references include *It Takes Two* (over 30 million units sold, GDD \$3.4), *A Way Out* (13 million units), and smaller independent co-op titles such as *Lovers in a Dangerous Spacetime* and *Unravel Two*, which achieved tens of thousands of peak concurrent players on Steam in their first weeks.

For an open beta of a new indie title without a pre-existing audience, a conservative but credible projection draws on the lower end of this range.

Beta registration estimate. We estimate **5 000–10 000 beta registrations** based on a targeted Steam wishlist campaign, social media outreach, and Discord community building prior to launch. This figure is consistent with the lower range of visible beta participation for indie two-player-only PC games.

Daily Active Users (DAU). Beta participants do not all play every day. Based on typical DAU-to-registered ratio for beta games (15–25%), we model:

$$\text{DAU} = 10,000 \times 0.20 = 2,000 \text{ players/day (nominal)}$$

Peak Concurrent Users (PCU). Peak concurrency during a beta is typically observed on weekends, in the first 2–3 weeks after launch. The ratio of PCU to DAU for co-op games is lower than for competitive games, as sessions require two players to agree to play simultaneously. We model a **PCU-to-DAU ratio of 50%**, giving:

$$\text{PCU} = 2,000 \times 0.50 = 1,000 \text{ concurrent players (peak)}$$

Since every session requires exactly two players, 1,000 concurrent players correspond to **500 concurrent game sessions** at peak load. This is the primary dimensioning target for the infrastructure.

Design ceiling. To allow for a positive viral scenario (successful streamer coverage, press attention), the architecture is over-provisioned to accommodate up to **5 000 concurrent players (2 500 sessions)** without any structural change to the backend.

Metric	Conservative	Optimistic ceiling / Design ceiling
Beta registrations	5 000	10 000
DAU (20% of registrations)	1 000	2 000
PCU (50% of DAU)	500	1 000
Concurrent sessions (PCU / 2)	250	500
Design ceiling (sessions)	2 500 (no architectural change required)	

Table 4: User base projection for the open beta. Infrastructure is dimensioned against the optimistic concurrent session count (500), with a 5x growth ceiling before scaling actions are required.

Per-Session Resource Profile

A single BBL session generates two distinct categories of load: real-time game traffic (routed through Steam SDR) and backend API traffic (routed to our persistence infrastructure). Only the latter is a direct infrastructure cost.

Session duration and structure. Sessions last 30–60 minutes (GDD §8.4.12). We model a mean session duration of **45 minutes**. Within a session, players trigger checkpoint writes approximately every 5–10 minutes (on room completion); we use a conservative interval of **7.5 minutes per checkpoint**. This yields approximately 6 checkpoint saves per session.

Backend API calls per session. Each session generates the following API interactions:

- 1 POST `/session/create` call (session start, host side).
- 1 POST `/session/join` call (session start, guest side).
- ≈ 6 POST `/session/save` calls (checkpoints).
- 1 POST `/session/end` call (session end).
- 1 GET `/save/{steamid}` call per player at game launch (2 total, but these are typically distributed across the day rather than concentrated at session start).

Total per-session API calls (excluding game-launch reads): approximately **9 API requests per session**.

Payload size.

- **Session create/join** requests: ≈ 0.5 KB (Steam ticket, session code if applicable). Response: < 1 KB (session token, save file metadata).
- **Checkpoint save** payload: the `save_files` row serialised as JSON, comprising sector unlock flags (bitmask), shared currency balance (integer), permanent upgrade levels (small JSON object), and last checkpoint ID. Estimated size: **< 2 KB per write**.
- **Session end** payload: identical structure to checkpoint save.

Real-time game traffic (Steam SDR). Each session exchanges NGO state-replication packets at 20 ticks/second. A conservative per-tick payload estimate (positions, HP, Bond Energy, animation states, enemy entity deltas) is 200–500 bytes. Bidirectional throughput per session:

$$500 \text{ B} \times 20 \text{ tick/s} \times 2 \text{ directions} \approx 20 \text{ KB/s per session}$$

This traffic is carried entirely by Steam’s SDR network and is not a cost factor for our infrastructure.

Aggregate Load Computation

API request rate. At 500 concurrent sessions, the steady-state API write rate from checkpoints is:

$$\frac{500 \text{ sessions}}{7.5 \text{ min} \times 60 \text{ s/min}} \approx \mathbf{1.1 \text{ req/s}}$$

Session lifecycle calls (create, join, end) are spread across the session boundaries. At 500 concurrent sessions, with a mean duration of 45 minutes, new sessions start at a rate of $500/45 \approx 11$ per minute, generating $11 \times 2/60 \approx 0.4$ lifecycle calls/second. Including all call types, the total API request rate at peak beta load is:

$$\text{Total API rate} \approx 1.1 + 0.4 \approx \mathbf{1.5 \text{ req/s (peak)}}$$

Under the design ceiling of 2500 sessions, the maximum API rate scales to ≈ 7.5 req/s,

still well within single-instance capacity.

Database write throughput. Each checkpoint write modifies one row in the `save_files` table (upsert by save file UUID). At 1.1 writes/second and < 2 KB per row:

$$\text{Write throughput} < 1.1 \times 2 \text{ KB} = \mathbf{2.2 \text{ KB/s}}$$

This is negligible. For reference, a `db.t3.micro` RDS instance is rated for thousands of lightweight transactions per second.

Database read throughput. The `GET /save/{steamid}` endpoint is called once per player at game launch. Assuming the 2 000 DAU spread uniformly over a 12-hour active window, the average read rate is $2,000/(12 \times 3600) \approx 0.05$ reads/second. During the first 30 minutes after beta launch (spike scenario), we model 500 simultaneous first-logins generating approximately 8 reads/second: still within single-instance capacity.

Steam SDR aggregate bandwidth. At 500 concurrent sessions at 20 KB/s each, total SDR relay traffic is:

$$500 \times 20 \text{ KB/s} = \mathbf{10 \text{ MB/s}}$$

This is carried by Valve’s infrastructure. At the design ceiling of 2 500 sessions, SDR traffic reaches 50 MB/s, still a trivial load relative to the scale of Valve’s global relay network.

Infrastructure Sizing Summary

Table 6 consolidates the per-component load estimate with the selected instance and its rated capacity.

Component	Instance / Service	Peak load (500 sessions)	Design ceiling (2 500 sessions)
REST API server	EC2 <code>t3.small</code> (2 vCPU, 2 GB)	≈ 1.5 req/s	≈ 7.5 req/s
Database writes	RDS PostgreSQL <code>db.t3.micro</code>	≈ 1.1 writes/s, < 2 KB each	≈ 5.5 writes/s
Database reads	RDS PostgreSQL <code>db.t3.micro</code>	≈ 0.05 reads/s (avg.) ≈ 8 reads/s (launch spike)	≈ 40 reads/s (launch spike)
Game traffic	Steam SDR (Valve)	≈ 10 MB/s	≈ 50 MB/s
Auth validation	Steamworks Web API (Valve)	< 0.3 calls/s	< 1.5 calls/s

Table 6: Infrastructure sizing against the open beta workload. Columns show the primary dimensioning target (500 sessions) and the design ceiling (2 500 sessions). Game traffic and authentication are handled by Valve infrastructure and impose no cost on our own backend.

Cost Projection

Table 8 provides a monthly cost estimate for the cloud infrastructure during the open beta, based on AWS EU (Frankfurt) on-demand pricing as of Q1 2026.

Service	Configuration	Estimated monthly cost
EC2 t3.small (API server)	1 instance, on-demand, 24/7	$\approx$ \$15–18
RDS PostgreSQL db.t3.micro	Single-AZ, 20 GB SSD storage, 24/7	$\approx$ \$13–16
Application Load Balancer	1 ALB, low traffic volume	$\approx$ \$18–20
Data transfer (outbound)	< 5 GB/month (API responses only)	< \$1
AWS CloudWatch	Basic metrics and alarms	$\approx$ \$2–5
Total		$\approx$ \$50–60 / month

Table 8: Estimated monthly AWS cost for the open beta infrastructure. Real-time game traffic (Steam SDR) is included in Steamworks access at no additional charge to the developer.

The total infrastructure cost of approximately \$50–60 per month is viable for an indie team throughout the duration of an open beta (typically 1–3 months). If the beta is extended or user numbers exceed the design ceiling of 2 500 concurrent sessions, the addition of a second EC2 instance behind the existing load balancer (approximately \$15–18/month additional) and an upgrade to `db.t3.small` (approximately \$26/month) would be sufficient to double capacity, bringing the total to approximately \$100–110/month: still a small operational expense relative to the marketing value of an extended beta.

5 General Architecture

Bound by Light adopts a **host-client architecture over Steam’s relay network**, complemented by a **lightweight cloud backend** for persistent data. The overall system is composed of three logical tiers, as illustrated in Figure 1.

- **Client Tier.** Two Unity game clients running on player machines. One client additionally acts as the authoritative game session host (listen-server model).
- **Platform/Network Tier.** Valve’s Steam infrastructure (authentication, Friend Pass, Steam Datagram Relay) acts as the network platform layer: it is the external entity through which both clients communicate, and from which we purchase services (auth, relay, social graph) as a customer.
- **Backend Tier.** A custom cloud-hosted service (REST API + PostgreSQL database) responsible exclusively for persistent state: session codes, save files, permanent upgrades, and per-player narrative logs.

This separation is deliberate. The game session itself (all real-time gameplay state: positions, health, Bond Energy, enemy states) is managed entirely within the listen-server instance on the host’s machine and transported through Steam SDR. Our own backend is never on the critical path for real-time gameplay, which prevents backend latency from affecting session responsiveness. The backend only receives calls at session boundaries and at checkpoints, where strict latency requirements do not apply.

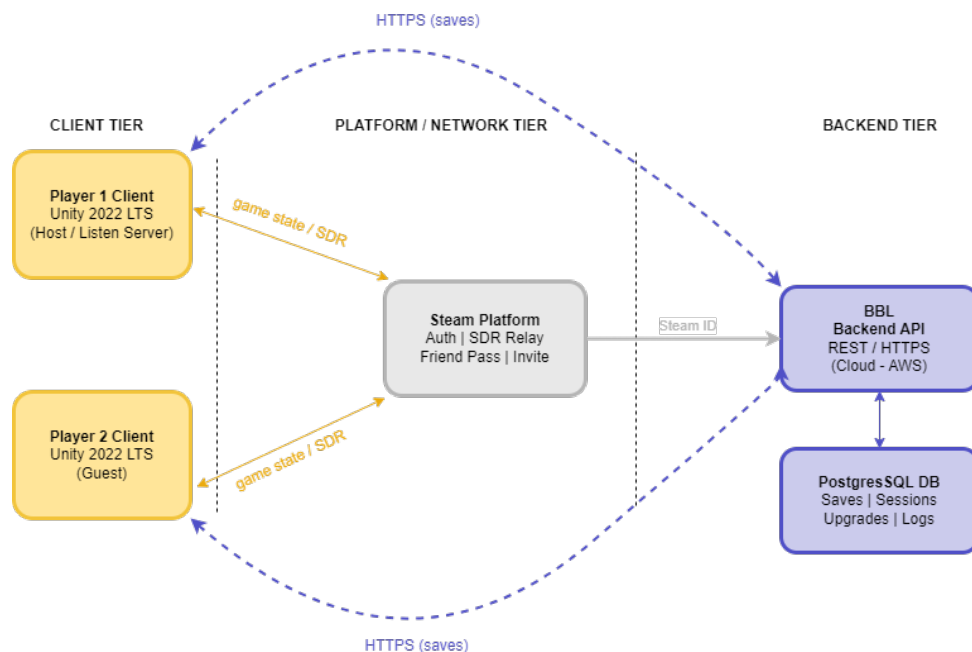


Figure 1: High-level architecture overview of *Bound by Light*. Solid coloured arrows represent real-time game traffic routed through Steam SDR. Dashed arrows represent HTTPS REST calls to the custom backend, occurring only at session boundaries and checkpoints.

The choice of the listen-server model over a dedicated game server is justified by the strict two-player constraint of BBL. A dedicated game server would be required to host all session logic regardless of whether players are present, multiplying infrastructure costs proportionally with concurrent sessions. With exactly two players per session, one of whom is already maintaining an active game process, the listen-server model amortises the session-hosting cost to zero on the infrastructure side. The sole trade-off is that a host disconnection terminates the session (GDD §8.9.8), which is an acceptable design decision given the short session structure and checkpoint system.

The Platform Tier (Steam) acts precisely as the “game network” described, an independent commercial entity whose infrastructure we use as a paying customer. Steam SDR provides relay PoPs (Points of Presence) globally that route traffic between clients even when direct P2P is unavailable due to NAT or firewall restrictions, handling the role of the frontend DMZ and authentication layer identified in the classic online gaming architecture model.

5.1 Frontend

The *frontend* of BBL is the combination of (a) the **game client application** running on the player’s machine, and (b) the **Steam Network frontend layer** that manages client authentication, connection establishment, and relay assignment before game-state packets begin flowing. Figure 2 illustrates the frontend components and the session setup flow.

Hardware

The hardware requirements for the client are defined in GDD §3.3 (see Table 1 of the GDD for full specifications). In summary:

- **Minimum:** Intel Core i5-6600 / AMD Ryzen 5 1600, 8 GB RAM, NVIDIA GTX 1050 / AMD RX 570 (4 GB VRAM), 8 GB storage, broadband internet, DirectX 11.
- **Recommended:** Intel Core i7-9700K / AMD Ryzen 7 3700X, 16 GB RAM, NVIDIA GTX 1660 / AMD RX 5600 XT (6 GB VRAM), 10 GB SSD, DirectX 12.

The host player’s machine bears a higher computational load than the guest, since it runs the authoritative game simulation alongside the rendering pipeline. The recommended hardware targets are calibrated accordingly.

Software

- **Unity 2022 LTS:** the game engine. A game is technically a *real-time interactive agent-based computer simulation* in which independent entities (agents) live and interact within a continuously ticking world; Unity provides the simulation core (physics, entity update loop, rendering), the scripting environment (C#), and the build pipeline for Windows. The 2022 LTS release is chosen for production stability over the beta lifecycle.
- **Unity Netcode for GameObjects (NGO):** the multiplayer framework. NGO runs

on top of Unity’s transport layer and implements the host-client topology natively. It handles object ownership, state replication, remote procedure calls (RPCs), and network variable synchronisation between the listen-server and the connected guest client.

- **Steam transport for NGO (Facepunch.Steamworks):** Unity NGO supports pluggable transport implementations. The Facepunch.Steamworks transport replaces the default UDP transport with Valve’s Steam Sockets API, routing all NGO traffic through the Steam Datagram Relay. This provides NAT traversal, relay fallback, and end-to-end encryption at the transport layer without any additional server-side relay code.
- **Steam Client (mandatory):** players must have Steam installed and be logged in. The Steam client handles download, update delivery, and local authentication. The game binary is distributed exclusively through Steam (Epic Games Store as secondary channel for future consideration).
- **Steamworks SDK:** the game communicates with the Steam platform for: user authentication (obtaining a verified Steam session ticket), Friend Pass invitation sending and validation, and in-game overlay integration (notifications, connection status).

Services

The following Steam platform services are consumed by the frontend layer:

- **Steam Authentication.** Upon game launch, the client requests a session ticket from the local Steam process. This ticket is sent to the BBL backend (§5.2) for verification via the Steamworks Web API, establishing a trusted session without the game ever handling passwords or personal data. The game identifies players exclusively by their Steam ID, which keeps BBL compliant with GDPR by not storing any personally identifiable information directly.
- **Friend Pass.** Valve’s Friend Pass (ISteamGameServerStats and the Steam Remote Play Together framework) allows one player to invite a friend who does not own the game. The host generates an invitation through the Steam overlay; the invited player joins through the Steam client. This maps directly to the “Friend Pass invite” flow described in GDD §8.8.
- **Session Code invitations.** Alongside Friend Pass, the host can generate a one-time alphanumeric session code via the BBL backend. The guest enters this code in the Join menu, the backend validates it and permanently links the two Steam accounts to the corresponding save file. This mechanism is implemented at the application layer (BBL backend), not the Steam layer.
- **Steam SDR: Relay Assignment.** When a session is created, the Steam Networking Sockets API selects the optimal relay path for the two clients. This selection considers geographic proximity and route latency, targeting the ≤ 100 ms round-trip objective set in §1. The relay path is renegotiated transparently if network conditions

change during the session.

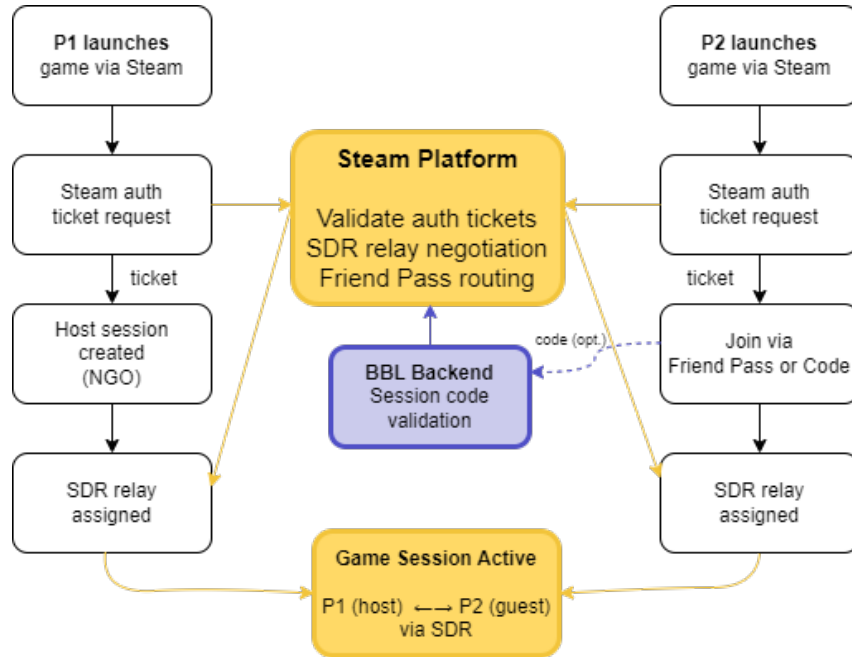


Figure 2: Frontend session setup flow. Both clients authenticate via Steam before connecting. The session-code path (dashed) involves the BBL backend only when a code-based join is used instead of a Friend Pass direct invitation.

5.2 Backend

The BBL backend tier is composed of two logical components, illustrated in Figure 3:

1. **Game Session Layer:** the listen-server running on the host player’s machine, which is the authoritative source of game state during an active session. This component is not a separately deployed service; it is an instance of the Unity game client operating in host mode under NGO.
2. **Persistence Backend:** a cloud-hosted REST API server and relational database, responsible for all data that must survive beyond a single session.

Game Session Layer (Listen Server)

Hardware. The listen-server runs on the host player’s PC, subject to the recommended hardware requirements stated in §5.1. No dedicated server hardware is provisioned for game sessions during the beta.

Software.

- **Unity Netcode for GameObjects, server mode.** When the host creates a session, NGO initialises the game in host mode. The host process maintains the authoritative game state: all entity positions, health values, Bond Energy, shared currency, room

state, and checkpoint flags. The guest’s inputs are received as remote messages, applied to the simulation, and the resulting state is replicated back to the guest client at a fixed network tick rate (target: 20 ticks/s, adjustable for performance).

- **Server-side validation.** Game-critical actions (room transitions, currency changes, boss death, checkpoint writes) are validated exclusively by the host-side simulation. The guest client sends input commands, not state mutations. This prevents a guest-side process manipulation from corrupting the shared game state, while still allowing both players to animate and predict locally.
- **Disconnection handling.** The listen server implements the disconnection policy defined in GDD §8.9.8: if the host disconnects, the session terminates and both clients are returned to the main menu; if the guest disconnects, the simulation pauses and a reconnection timer starts. Reconnection uses the same Steam SDR session slot; if the timer expires, both clients resume from the last persisted checkpoint.

Services. The listen server communicates with the persistence backend over HTTPS at three defined points: (a) session start, to load the shared save file associated with the pair’s linked accounts; (b) checkpoint reached, to persist the current game state; (c) session end, to write final progression data (currency balance, permanent upgrades earned, narrative logs).

Persistence Backend

Hardware. The persistence backend is hosted on AWS (Amazon Web Services), using managed services to reduce operational overhead:

- **API Server:** AWS EC2 `t3.small` instance (2 vCPU, 2 GB RAM), running inside a Virtual Private Cloud (VPC) with a load balancer and auto-scaling group for the beta ramp-up. For the initial open beta, a single instance is sufficient; a second standby instance can be activated manually if user numbers grow beyond the estimated workload (see §5.3).
- **Database:** AWS RDS PostgreSQL `db.t3.micro` (2 vCPU, 1 GB RAM). Given the very low write throughput required (checkpoint-triggered saves), this instance size is well within capacity for the beta.
- **Network:** AWS Application Load Balancer in front of the API tier; database accessible only within the VPC (no public endpoint). TLS 1.3 on all external-facing endpoints.

Software.

- **REST API server:** Python (FastAPI framework), chosen for development speed and the team’s existing familiarity. Exposes the following endpoints over HTTPS:
 - `POST /session/create`: validates Steam auth ticket, generates or retrieves the save file for the host’s account.

- **POST /session/join:** validates Steam auth ticket, validates session code (if provided), permanently links guest account to save file.
 - **POST /session/save:** writes checkpoint data (game state, currency, upgrade flags) to the database.
 - **POST /session/end:** finalises session, writes end-state.
 - **GET /save/{steamid}:** retrieves the save file(s) associated with a Steam account on game launch.
- **PostgreSQL database schema (summary):**
 - **players:** Steam ID, creation timestamp, game version at first login.
 - **save_files:** UUID, player1_steamid, player2_steamid (nullable until linked), last checkpoint, sector unlock flags, shared currency, permanent upgrade levels.
 - **narrative_logs:** save_file UUID, steam_id (per-player), boss index, log type (story log / soul fragment vision), content hash. The log content itself is stored client-side; only its existence is tracked centrally to prevent re-delivery.
 - **Session code table:** one-time alphanumeric tokens stored with a 48-hour expiry. Upon use, the token is invalidated and the two accounts are permanently linked. This is the technical implementation of the game rule that “using the code permanently links the two players’ accounts to that specific save file” (GDD §8.9.8).

Services.

- **Steamworks Web API:** the backend calls Valve’s `AuthenticateUserTicket` endpoint to verify Steam session tickets received from clients, confirming that each connecting user is a valid, active Steam account holder. The game company is a customer of the Steam network; Steam provides this service as part of the game’s Steamworks registration.
- **AWS CloudWatch:** operational monitoring (CPU, memory, request latency, error rates) for the API server and database during the beta. Alarms are configured for anomalous error rates or sustained high CPU, which would indicate a capacity scaling event.

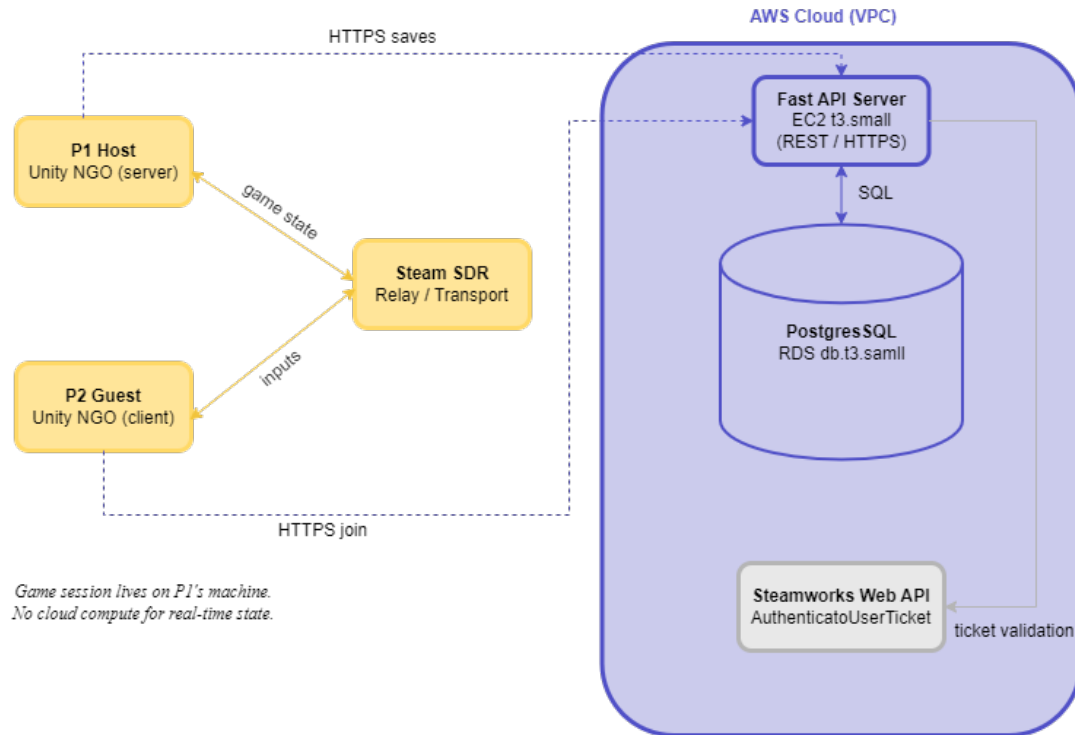


Figure 3: Backend architecture of *Bound by Light*. Real-time game state (solid arrows) is exchanged exclusively between the two Unity clients via Steam SDR. The cloud backend (dashed arrows) is only invoked at session start, checkpoints, and session end.

Database Schema - ER Diagram

Figure 4 shows the entity-relationship diagram of the PostgreSQL persistence layer. The schema is intentionally minimal: four tables cover all persistence requirements identified in §5.2. The **save_files** table is the central entity, permanently linking two **players** rows once a session code is consumed. **narrative_logs** stores per-player story records without exposing them to the partner, enforcing the information-asymmetry rule defined in GDD §8.9.7 at the data layer.

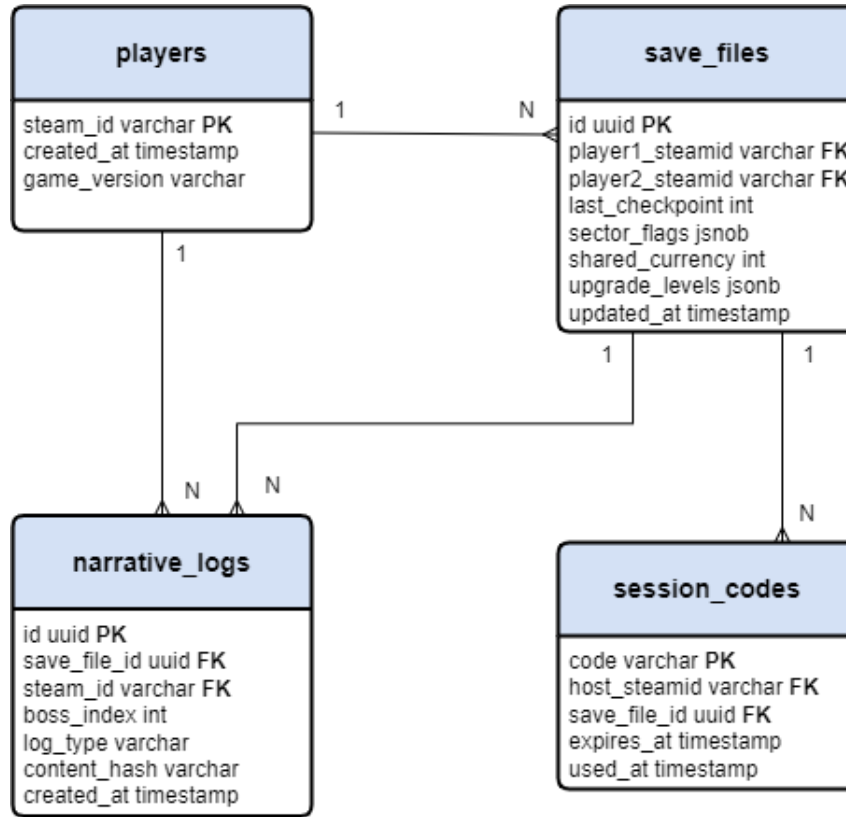


Figure 4: Entity-Relationship diagram of the BBL PostgreSQL schema. **PK** = Primary Key; **FK** = Foreign Key. Cardinalities are shown on each relationship arc.

5.3 Workload Capacity

The workload analysis distinguishes two components: (a) the real-time game session traffic, which is routed through Steam SDR and therefore does not burden our own infrastructure; and (b) the backend API and database load, which depends on player session behaviour.

Target Workload Estimate

User baseline. Bound by Light targets a beta audience consistent with an indie cooperative PC game. Comparable two-player-only titles validated on Steam include *It Takes Two* (30M+ units, GDD §3.4) and *A Way Out* (13M+ units), but these are post-launch figures for commercially successful titles. A conservative open-beta estimate draws from the observation that even modest co-op indie titles on Steam can sustain 500–2 000 concurrent players during a beta weekend. We design for a **peak concurrent player count of 1 000 players (500 active sessions)** as the dimensioning target for the beta, with a growth path to 5 000 players (2 500 sessions) without architectural change.

Session duration and checkpoint frequency. Sessions last 30–60 minutes (GDD §8.4.12). Checkpoints are triggered each time players complete a room section (approximately every 5–10 minutes per session). We model one checkpoint-save API call per 7.5 minutes per session.

Backend API throughput. At 500 concurrent sessions with one save call per 7.5 minutes:

$$\frac{500 \text{ sessions}}{7.5 \text{ min} \times 60} \approx 1.1 \text{ API saves/second (peak)}$$

Session-create and session-join calls occur once per session; over a one-hour period with 500 active sessions, they account for fewer than 10 calls/minute. The total API request rate is well below 5 requests/second at peak beta load. This is comfortably within the capacity of a single `t3.small` EC2 instance, which can handle hundreds of concurrent HTTPS connections.

Database write load. Each checkpoint write affects one row in `save_files` (typically < 2 KB of JSON-serialised state). At 1.1 saves/second and an average row size of 2 KB, the database write throughput is approximately 2.2 KB/s, negligible for an RDS instance. Read operations (session load on startup) are equally infrequent.

Real-Time Game Traffic (Steam SDR)

Each BBL game session involves two clients exchanging NGO state-replication packets at 20 ticks/second. A conservative estimate for per-session bandwidth:

- **State update packet size:** approximately 200–500 bytes (positions, HP, animation state, Bond Energy, enemy entity deltas).
- **Bidirectional throughput per session:** $500 \text{ B} \times 20 \text{ tick/s} \times 2 \approx 20 \text{ KB/s}$.

At 500 concurrent sessions, total game traffic is approximately $500 \times 20 \text{ KB/s} = 10 \text{ MB/s}$. This traffic is carried entirely by Steam’s SDR relay network and does not flow through our backend. Valve dimensions SDR capacity to support millions of concurrent game sessions across their platform, making this volume trivially absorbable. Our infrastructure is entirely decoupled from this load.

Session Lifecycle - Sequence Diagram

Figure 5 illustrates the complete interaction between all system components across the five phases of a BBL session: setup, relay establishment, gameplay loop, checkpoint, and session end. The diagram makes explicit that the two communication channels, real-time game traffic (NGO via Steam SDR) and persistent state management (HTTPS to the BBL backend), are fully independent and never share the same path.

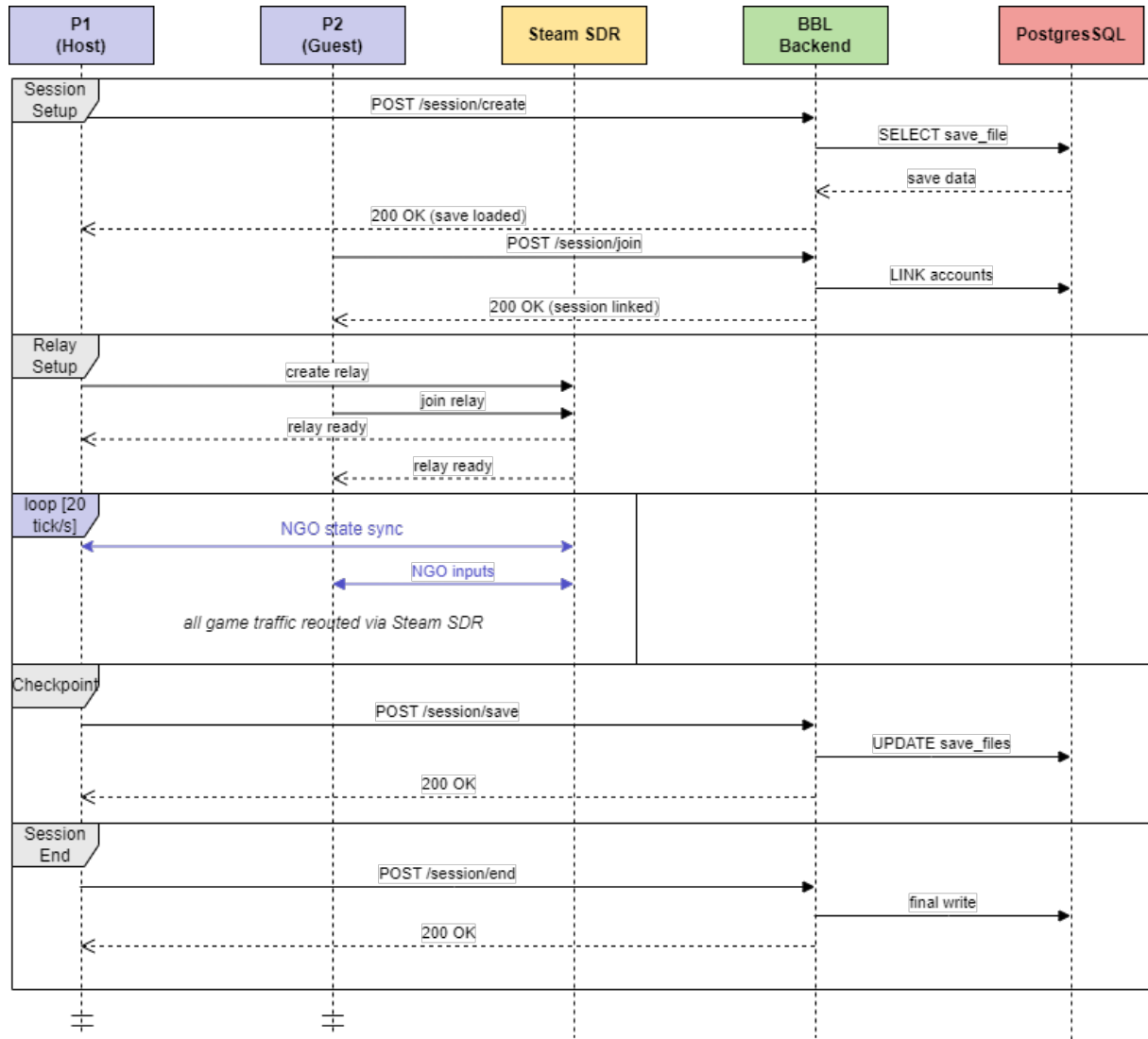


Figure 5: UML sequence diagram of a complete BBL session. Solid arrows are requests; dashed arrows are responses. The gameplay loop (blue) runs at 20 ticks/second via Steam SDR and has no interaction with the backend. Backend calls occur only at session boundaries and checkpoints.

Capacity Justification

The infrastructure holds the intended workload for the following reasons:

1. **Real-time traffic is externalised.** By routing all game-session traffic through Steam SDR, we remove the most demanding and latency-sensitive component from our own infrastructure entirely. This is consistent with the architectural principle that the game company is a *customer* of the game network infrastructure, not its operator.
2. **Backend load is session-boundary-only.** The REST API is invoked at a rate proportional to the number of sessions divided by the session duration, not at the game-tick rate. A session of 45 minutes with 6 checkpoints generates approximately 8 API calls total. Even at 500 concurrent sessions, this is below 2 API requests/second, an order of magnitude below what a `t3.small` can sustain.

3. **Database access is point-in-time, not streaming.** Unlike an MMO backend that must update a shared world state continuously, BBL’s database stores discrete snapshots. Write conflicts do not occur (each save file is owned by exactly one player pair). No locking contention or clustering is needed at beta scale.
4. **Horizontal scaling is available.** Should the beta exceed projected user numbers, the API tier can be scaled horizontally by adding EC2 instances behind the load balancer without any schema or code change. The database can be migrated to a larger RDS instance class with a few minutes of downtime in a maintenance window.
5. **Network latency is delegated to Valve.** Steam SDR’s global relay network selects the lowest-latency path between two clients automatically. Valve’s business incentive as a commercial platform operator is to maintain a high-quality relay network across all titles on their platform, an interest that aligns directly with our latency target. We do not need to operate our own relay infrastructure.

Workload Capacity - Visualisation

Figure 6 provides a visual representation of the data presented the Table below, plotting estimated beta peak load against provisioned capacity for each infrastructure component. The logarithmic scale is necessary given the order-of-magnitude differences between load and capacity, which reflect a deliberately conservative provisioning strategy suited to the unpredictable ramp-up patterns of an open beta.

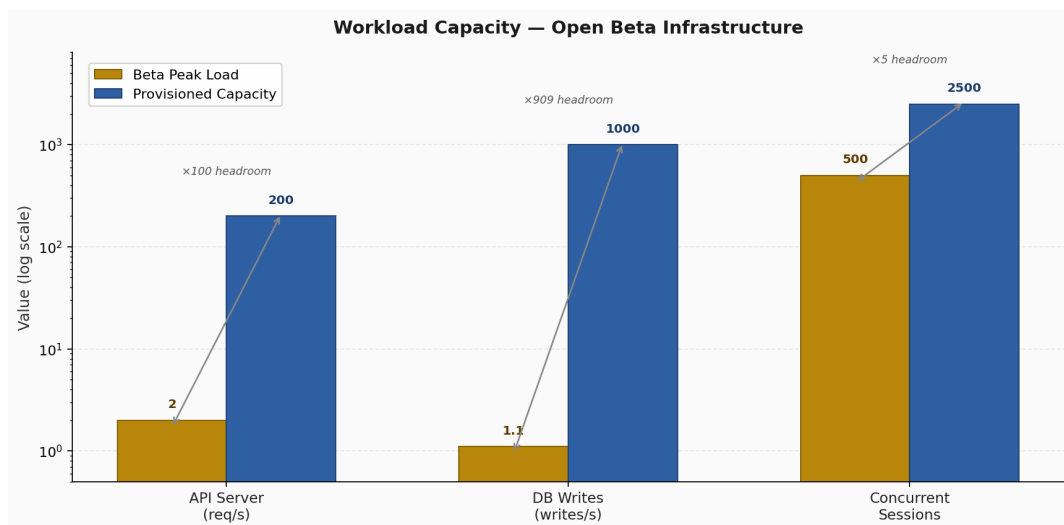


Figure 6: Comparison of estimated beta peak load (gold) and provisioned capacity (blue) for each backend infrastructure component. The API server and database show headroom of two to three orders of magnitude; concurrent sessions are bounded by the design ceiling of 2 500 rather than by hardware limits. Y-axis is logarithmic.

The following table summarises the dimensioning of the backend infrastructure against the expected beta workload.

Component	Beta peak workload	Provisioned capacity
API server (EC2 t3.small)	≈ 2 req/s, < 100 ms response	~ 200 req/s sustainable; $\times 4$ headroom
PostgreSQL (RDS db.t3.micro)	≈ 1.1 writes/s, < 2 KB each	Handles thousands of writes/s; $\times 1000$ headroom
Game session traffic	≈ 10 MB/s across all sessions	Routed via Steam SDR (unlimited from our perspective)
Concurrent sessions	500 target, 2 500 design ceiling	Stateless API; add instances behind ALB to scale
Authentication (Steamworks Web API)	< 0.3 ticket validations/s	Valve service; no capacity limit on our side

Table 10: Workload capacity summary for the open beta.

5.4 Network Synchronization and State Management

The host-client architecture described in §5 requires a precise definition of which game state is authoritative, how it is propagated to the guest client, and how conflicts between locally predicted state and server-authoritative corrections are resolved. This section documents the synchronisation model adopted in *Bound by Light* using Unity Netcode for GameObjects (NGO).

Authority Model

NGO implements a **server-authoritative model** in which the host process is the sole source of truth for all game state. The guest client never writes to shared state directly: it sends input commands to the host, which applies them to the simulation, and replicates the resulting state back. This maps cleanly to the listen-server topology of BBL.

Two complementary mechanisms are used to propagate state:

- **NetworkVariable<T>**: continuous, replicated state that the host updates and NGO synchronises to connected clients automatically on change (with configurable dirty-checking). Used for all quantities that must remain consistent across both clients at all times.
- **Remote Procedure Calls (RPCs)**: discrete events that are reliable and ordered.

ServerRpc methods are called by a client and executed on the host; **ClientRpc** methods are called by the host and executed on one or all clients. Used for one-shot game events where a state transition must be acknowledged, not just observed.

State Ownership Map

Table 11 defines which BBL game state variables are carried by each mechanism and which object owns write authority.

Table 11: State ownership and synchronisation mechanism for each BBL game state variable. All write authority ultimately resolves to the host process, with the exception of client-authoritative player movement (reconciled by the host).

State variable	Mechanism	Write authority	Notes
Player position / velocity	NetworkTransform	Owner client	Client-authoritative transform with server reconciliation (see below).
Player health	NetworkVariable <int>	Host (server)	Damage is applied exclusively by host-side collision logic.
Bond Energy bar	NetworkVariable <float>	Host (server)	Single shared value; both clients read it. Drain/recharge triggered by host-validated tether state.
Tether link state	NetworkVariable <bool>	Host (server)	Encodes whether the tether is active, stretched, or broken. Critical for the core mechanic; authoritative on host.
Shared currency	NetworkVariable <int>	Host (server)	Modified only on host; read-only on guest.
Enemy entity state	NetworkVariable (per entity)	Host (server)	Enemy AI runs exclusively on the host. Position, HP, and animation state are replicated.
Room transition trigger	ClientRpc	Host (server)	Emitted by host when both players are in the exit zone. Transitions all clients simultaneously.
Checkpoint reached	ClientRpc + backend call	Host (server)	Host emits the RPC to notify both clients, then calls <code>POST /session/save</code> asynchronously.

State variable	Mechanism	Write authority	Notes
Narrative log unlock	TargetRpc	Host (server)	Sent to a specific client only, enforcing the information-asymmetry rule (GDD §8.9.7).

Client-Side Prediction and Reconciliation

Player movement is the only state variable handled with **client-authoritative prediction**. Without prediction, the guest player would experience input lag equal to the full round-trip delay (up to 100 ms), making movement feel unresponsive. The mechanism works as follows:

1. The guest client applies its own movement input locally and immediately advances its predicted position (speculative execution).
2. The same input is sent to the host as a **ServerRpc** tagged with a client-side timestamp and sequence number.
3. The host applies the input to the authoritative simulation and returns the resulting position via **NetworkTransform** replication.
4. On receiving the host correction, the guest compares it against its predicted state. If the discrepancy exceeds a configurable threshold (default: 0.1 m), the guest **snaps to the corrected position**; otherwise, it smoothly interpolates over one to two frames to avoid visible jitter.

The host player does not require prediction, as it runs the authoritative simulation locally and perceives zero network delay to the game state.

Tether Mechanic Synchronisation

The tether is the mechanically and technically most sensitive state in BBL. Its correctness affects Bond Energy drain, movement constraints, and combat outcomes simultaneously. The synchronisation strategy is as follows:

- **Tether geometry** (rope endpoints, stretch length) is computed exclusively on the host using the authoritative positions of both players. The guest’s predicted position is *not* used for tether physics; only the host-validated position is.
- **Tether state transitions** (active, stretched, broken) are server-authoritative events emitted via **ClientRpc** to both clients simultaneously, ensuring that Bond Energy drain starts and stops at the same logical tick on both machines.
- **Visual representation** of the tether on the guest client is rendered using the replicated endpoint positions with local interpolation for smoothness; the visual is decorative and may lag the authoritative state by one to two network ticks without affecting gameplay correctness.

Network Tick Rate and Interpolation

NGO state replication runs at a fixed **network tick rate of 20 Hz** (one synchronisation pass every 50 ms). This is a deliberate trade-off between bandwidth and responsiveness:

- At 20 Hz, each replicated entity update arrives every 50 ms on average. With a 100 ms RTT budget, the guest observes state that is at most one to two ticks old under normal conditions.
- Entities whose state changes between ticks are **interpolated** on the receiving client: NGO’s `NetworkTransform` interpolates position and rotation between the last two received snapshots, producing smooth visual motion even at 20 Hz replication.
- For fast-moving entities (projectiles, dashing player), a higher per-object tick override may be applied selectively without raising the global tick rate.

The 20 Hz tick rate results in a per-session bandwidth of approximately $500 \text{ B} \times 20 \text{ tick/s} \times 2 = 20 \text{ KB/s}$, as computed in §5.3. Raising the tick rate to 30 Hz would increase this to 30 KB/s per session, still well within Steam SDR capacity, and may be evaluated during the beta based on player feedback on movement responsiveness.

5.5 Production Migration Path: from Open Beta to Commercial Release

The architecture described in §5–§5.3 is explicitly designed for the open beta and the constraints that come with it: a small development team, a limited initial user base, and the need to minimise both operational overhead and infrastructure costs during an exploratory phase. Several of those constraints relax or disappear when the product moves toward a commercial release. This section documents the anticipated migration from the beta listen-server model to a **dedicated server architecture**, and identifies which existing components remain unchanged.

Limitations of the Beta Architecture in Production

Three limitations of the listen-server model become progressively more significant as the player base grows:

Host-side latency asymmetry. In the listen-server topology, Player 1 (host) maintains the authoritative simulation on their own machine and therefore experiences 0 ms network latency to the game state. Player 2 (guest) perceives the world through a full round-trip delay, typically 40–150 ms depending on geographic distance and connection quality. In a cooperative game this asymmetry does not create unfairness between the two players (they share objectives), but it does create a qualitatively inferior experience for the guest and may cause perceptible input lag during fast-paced combat sequences. At beta scale this is an accepted trade-off; at commercial scale it is a competitive disadvantage relative to titles running dedicated servers.

Session lifetime coupled to host connection. Under the current design, a host disconnection terminates the session for both players (GDD §8.9.8). This is acceptable in a beta with a forgiving player base but is likely to generate negative reviews at commercial launch, particularly if host disconnections correlate with losing hard-earned in-session progress. A dedicated server decouples session lifetime from any individual player’s connection.

Security and anti-cheat. The host player runs the authoritative simulation process on their own machine. While the cooperative nature of BBL makes adversarial exploitation between the two players unlikely, a compromised host process could still corrupt shared save state or exploit the information-asymmetry mechanic by reading memory that should be private to the guest. A dedicated server running on infrastructure under the developer’s control eliminates this attack surface entirely.

Target Production Architecture

The production target replaces the listen-server with a **managed dedicated game server fleet**, while leaving the persistence backend unchanged. Two industry-standard options are evaluated:

AWS GameLift. Amazon GameLift is a managed service for deploying, operating, and scaling dedicated game servers on EC2 fleets. The server binary (a headless Unity build) is uploaded to GameLift as a *build artifact*; GameLift manages instance lifecycle, session placement, and geographic routing across its global fleet of Regions. BBL would use the GameLift *FlexMatch* matchmaking system solely for the two-player lobby step (create/join session), and the GameLift SDK embedded in the server binary for session lifecycle notifications.

Agones on Google Kubernetes Engine (GKE). Agones is an open-source dedicated game server framework built on Kubernetes, maintained by Google and Ubisoft. Rather than a managed service, it gives the team full control over the server runtime: the Unity headless binary runs as a Kubernetes *Pod*, and Agones manages its lifecycle (allocation, health-checking, draining). This option is more complex to operate but avoids vendor lock-in and can be cost-optimised more aggressively using spot/preemptible instances. It is worth evaluating if user numbers grow to the point where GameLift pricing becomes a primary budget concern.

The architecture diagram in Figure 7 illustrates the production topology in contrast to the beta topology.

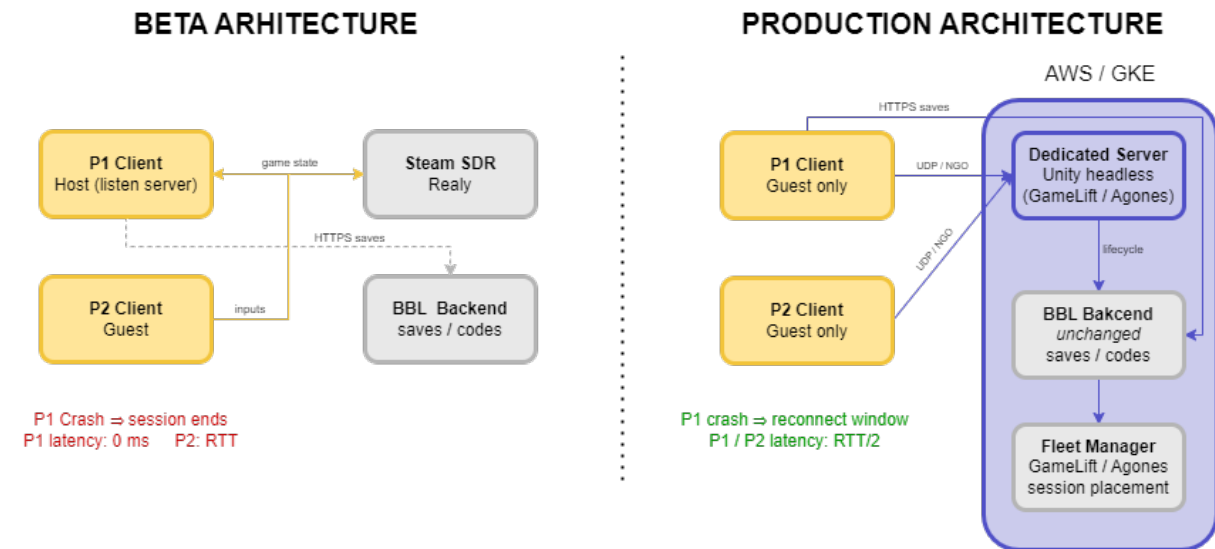


Figure 7: Comparison between the beta listen-server topology (left) and the production dedicated-server topology (right). The persistence backend (BBL Backend) remains unchanged in both configurations. The main differences are the introduction of a neutral server process, the symmetric latency experienced by both players, and the decoupling of session lifetime from the host connection.

What Changes with the Migration

Table 12 summarises which components are affected and which are carried forward unchanged.

Table 12: Component-level comparison between beta and production architectures. Components marked as *unchanged* require no rework and can be reused directly in the production deployment.

Component	Beta (listen server)	Production (dedicated server)
Unity client build	Single build: host mode + guest mode, Facepunch.Steamworks transport	Single client build (guest mode only); separate headless server build
Game session host	P1's player machine	GameLift EC2 instance or Agones Kubernetes Pod
Network transport	Steam Sockets (SDR); NAT traversal included	Standard UDP via Unity Transport or custom; NAT traversal handled by fleet manager

Continue in the next page

Table 12 – Continue from the previous page

Component	Beta (listen server)	Production (dedicated server)
Latency profile	Asymmetric: $P1 = 0$ ms, $P2 = \text{full RTT}$	Symmetric: both players at $\approx \text{RTT}/2$ to nearest server region
Host disconnection	Session terminates for both players	Session survives; disconnected player has a reconnection window
Session placement	Implicit: $P1$'s machine, wherever they are	Geographic placement by GameLift FlexMatch / Agones allocator to minimise average RTT for the pair
BBL Backend (REST + DB)	Handles saves, session codes, auth	Unchanged. Identical schema and endpoints; only the caller changes from $P1$'s client to the dedicated server process
Steam integration	Auth, Friend Pass, SDR transport, invitations	Auth, Friend Pass, invitations retained; SDR transport replaced by dedicated server transport
Anti-cheat surface	Host process on $P1$'s machine (uncontrolled environment)	Server process on developer-controlled infrastructure
Infrastructure cost	Zero (compute on $P1$'s machine)	$\approx$ \$0.03–0.08 per session-hour (GameLift on-demand pricing)

Migration Trigger Criteria

The migration from beta to production architecture is recommended when any of the following conditions are met:

- **User scale:** peak concurrent sessions consistently exceed 2 500 (the design ceiling of the beta architecture identified in §5.3), suggesting sustained commercial traction

that justifies dedicated server costs.

- **Player experience:** post-beta player feedback identifies host disconnection or latency asymmetry as primary pain points in reviews or support tickets.
- **Security:** any confirmed instance of host-side memory manipulation affecting shared game state or narrative log integrity.
- **Platform expansion:** if a console port is developed, Steam SDR is unavailable as a transport on consoles; a platform-agnostic dedicated server architecture becomes mandatory.

The persistence backend (REST API and PostgreSQL schema) is designed to be transport-agnostic: it accepts HTTPS calls from any authenticated caller, whether that is a Unity client or a dedicated server process. No schema migrations or API changes are anticipated for the production migration, which significantly reduces the engineering cost of the transition.

6 Quality Assurance and Testing Strategy

Online games introduce testing challenges that do not arise in single-player software: network-dependent behaviour, state synchronisation correctness, and failure modes that only manifest under specific connectivity conditions. This chapter defines the testing strategy adopted for *Bound by Light*, structured as a three-tier pyramid: unit tests, integration tests, and system-level network tests.

Testing Pyramid

- **Unit tests** verify isolated logic with no network or database dependency. Fast to execute, run on every commit.
- **Integration tests** verify the interaction between the REST API and the PostgreSQL database. Run on every pull request to the main branch.
- **System tests** verify end-to-end behaviour of the full system under controlled network conditions. Run manually before milestone builds and after any change to the synchronisation layer.

Unit Testing

Game logic (Unity Test Framework, EditMode). Game rules that can be expressed as pure functions are tested in Unity’s EditMode runner, which executes without starting the Unity player loop and requires no network context. Test coverage targets include:

- **Tether constraint logic:** given two positions and a tether length, assert correct output state (active, stretched, broken) and Bond Energy delta.
- **Bond Energy drain/recharge:** assert that drain rate, recharge rate, and depletion/recovery thresholds match the maximum values defined in the GDD and that the energy state transitions occur correctly.
- **Currency and upgrade arithmetic:** assert that reward calculation, upgrade cost deduction, and save-file serialisation are consistent and reversible (round-trip test).
- **Session code generation:** assert uniqueness properties of the token generator, correct 48-hour expiry assignment, and rejection of an already-consumed token.

REST API logic (pytest). The FastAPI backend is tested using `pytest` with a dedicated test database (SQLite in-memory for speed, PostgreSQL for pre-release validation). The `FastAPI TestClient` allows HTTP-level testing without a running server process. Test cases cover:

- **Happy-path flows:** valid Steam ticket, valid session code, correct save-file creation and retrieval.
- **Authentication rejection:** malformed ticket, expired ticket, unknown Steam ID. Assert HTTP 401 in all cases.

- **Session code lifecycle:** code generation, first-use linking, duplicate-use rejection, expiry rejection.
- **Checkpoint idempotency:** assert that two identical `POST /session/save` calls with the same payload produce the same database state (upsert semantics).
- **Boundary conditions:** maximum payload size, missing fields, malformed JSON. Assert HTTP 422 with a structured error body.

Integration Testing

Integration tests verify the API and database together against a real PostgreSQL instance, using `pytest` with `testcontainers-python` to spin up an ephemeral containerised database on each test run. This ensures that ORM queries, schema constraints, and foreign-key relationships behave correctly, and that the test database schema is always in sync with the production schema via Alembic migrations.

Key integration scenarios:

- **Session linking end-to-end:** host creates session, guest joins with code, assert that `save_files.player2_steamid` is correctly populated and the code is invalidated.
- **Checkpoint persistence:** create a session, write three checkpoint saves with incremental state, load the save file, assert the latest checkpoint is returned.
- **Concurrent checkpoint writes:** simulate two `POST /session/save` requests for the same save file arriving within the same database transaction window. Assert that no write is lost and the later timestamp wins (last-write-wins semantics, as save files are per-pair and concurrent writes from a single session are not expected in normal operation).
- **Narrative log isolation:** assert that `GET /save/{steamid}` for player 1 does not return narrative log entries belonging to player 2 of the same session.

System and Network Testing

System tests exercise the full stack under controlled network conditions using two Unity editor instances on the same machine (or two machines on a LAN) with Unity’s built-in **Network Simulator** package, which injects configurable packet delay, jitter, and loss into the NGO transport layer.

Baseline connectivity test.

- Assert that a host and guest can establish a session, reach a checkpoint, and complete a room transition with zero simulated network impairment. Verifies the happy path of the sequence diagram in Figure 5.

Latency stress test.

- Apply 80 ms one-way delay (160 ms RTT) and assert that: player movement remains responsive with prediction active; Bond Energy values converge within two network ticks after a tether state change; no desync is observed over a 10-minute session.
- This test directly validates the ≤ 100 ms RTT design target stated in §1 against a near-limit network condition.

Packet loss test.

- Apply 5% uniform packet loss and assert that NGO's reliable delivery mechanism (over Steam Sockets) retransmits lost packets without observable game state divergence. Assert that no `NetworkVariable` update is permanently lost.

Host disconnection test.

- Force-kill the host process mid-session. Assert that: the guest client detects the disconnection within the NGO timeout window (default: 10s); the guest is returned to the main menu with an appropriate error message; no orphaned session record remains active in the database.

Guest disconnection and reconnection test.

- Disconnect the guest client mid-session and reconnect within the reconnection timer window. Assert that: the host simulation pauses correctly; the guest re-joins the same NGO session using the same Steam SDR slot; game state on the guest matches the host's authoritative state at the moment of reconnection.
- Repeat with reconnection *after* timer expiry. Assert that both clients resume from the last persisted checkpoint, not from the in-memory state at disconnection time.

Checkpoint recovery test.

- Write a checkpoint, then simulate a crash of both clients. Relaunch, load the save file from the backend, and assert that the game state matches the last written checkpoint exactly (sector unlocks, currency balance, upgrade levels, narrative log entries).

Performance Validation

The following metrics are monitored during system tests and during the open beta via AWS CloudWatch and Unity Profiler:

Metric	Target	Measurement tool
Round-trip latency (P1 to P2)	$\leq$ 100 ms (95th percentile)	Unity Network Simulator; Steam Networking Sockets API stats
API response time	$\leq$ 200 ms (95th percentile)	CloudWatch <code>TargetResponseTime</code> ; FastAPI middleware timer
Checkpoint write latency (DB)	$\leq$ 50 ms per write	CloudWatch <code>RDS WriteLatency</code> metric
API error rate (5xx)	$< 0.1\%$ of requests	CloudWatch <code>HTTPCode_Target_5XX_Count</code>
Session state divergence	Zero divergences per 10-minute test session	Custom NGO divergence detector (hash of authoritative state vs. replicated state on guest)

Table 14: Performance validation targets and measurement tools for the BBL open beta. Metrics that breach their target threshold during the beta trigger an investigation before the next release build is tagged.

7 Physical Services

This chapter describes the physical infrastructure required to establish and operate the Juggernaut development studio for the duration of the project.

Office Space

The studio requires a dedicated workspace accessible to all team members. The location must be served by public transport and situated in an area with affordable commercial rents. The optimal location is the **Sesto San Giovanni** municipality (Metropolitan City of Milan), within walking distance of the *Sesto 1° Maggio FS* stop on Metro Line 1 (MM1). This area combines reasonable rent levels with good rail and metro connectivity from central Milan.

The ideal space is a **soundproofed loft** of sufficient size to accommodate a team of three, with room to expand to five workstations if additional hires are made. A well-ventilated basement unit is an acceptable alternative, provided natural or mechanical ventilation meets regulatory standards for continuous occupancy. Break areas are not required within the office; team members will use external venues for this purpose.

Hardware and Equipment

The following equipment is required at studio setup:

- **5 high-end desktop workstations.** Three units for the founding team members, two additional units reserved for future hires. Each machine must meet or exceed the recommended specifications for Unity development and 3D asset creation: a modern multi-core CPU (Intel Core i9 or AMD Ryzen 9 class), 32 GB RAM, a dedicated GPU with at least 8 GB VRAM (NVIDIA RTX series), and NVMe SSD primary storage.
- **2 pen display tablets.** Required for the artists on the team for concept work, texture painting, and UI design. Screen-integrated models (e.g., Wacom Cintiq class or equivalent) are preferred over screenless tablets for professional 2D and 3D work.
- **Standard peripherals.** Monitors (one per workstation, two for the art stations), keyboards, mice, and headsets. These do not need to be premium models; mid-range peripherals are sufficient.
- **Network infrastructure.** A commercial-grade broadband connection with a local switch for wired connectivity. A stable, low-latency connection is required for version control synchronisation and remote playtesting.

Software Subscriptions

The following software licenses and subscriptions are required for development:

- **Unity Pro** (or Unity Plus, depending on revenue threshold): the game engine and build pipeline.
- **Alienbrain**: version control and asset management system, chosen for its native support for large binary game assets.
- **Adobe Photoshop** (Creative Cloud subscription): 2D image editing for UI assets, textures, and concept art.
- **Autodesk Blender** (free, open-source): primary 3D modelling tool.
- **ZBrush** (Maxon subscription): high-resolution sculpting for character and enemy models.
- **Substance 3D Painter** (Adobe subscription): PBR texture authoring for 3D assets.
- **FMOD Studio** (free for indie): spatial and procedural audio authoring. The FMOD indie license is royalty-free for projects below a defined annual revenue threshold.
- **AWS services**: cloud backend infrastructure as described in §5.2, billed monthly on usage.
- **Unity Asset Store plugins**: as required during development to accelerate implementation of specific systems (physics helpers, shader utilities, UI frameworks). Evaluated and purchased on a case-by-case basis.

8 Delivery

8.1 Delivery Platforms

Bound by Light will be distributed through **Steam** as the primary and mandatory platform (GDD §3.3). Steam is the dominant distribution channel for PC games in the cooperative indie segment and provides the authentication, social graph, and relay infrastructure on which the game’s online architecture is built (see §5.1). A Steam account is a hard runtime requirement for all players.

Epic Games Store is under evaluation as a secondary distribution channel for the post-beta commercial release. The decision will be made based on beta performance metrics and Epic’s developer terms at the time of commercial launch.

Future platform considerations, contingent on commercial success and additional porting resources, include PlayStation 4 and PlayStation 5, Xbox (via Game Pass), and Nintendo Switch and Switch 2. These ports are out of scope for the current TDD; a platform-specific technical addendum would be required for each, as the Steam-native network architecture described in this document is not directly portable to console platforms.

8.2 Estimated Delivery Time

Academic demo (milestone). A playable demo encompassing the core game loop, the tether mechanic, at least one section of a plausible sector, and the full session management flow (session code, checkpoint, reconnection) is scheduled for **19 May 2026**.

Commercial release. The full commercial release of *Bound by Light* is planned for **Q3 2028**. This timeline accounts for the following post-demo production phases: completion of all game sectors and boss encounters, full audio production (soundtrack and spatial sound effects recorded and integrated via FMOD), comprehensive playtesting and QA cycles, asset finalisation for all map environments, and an extended open beta period prior to launch. The schedule includes buffer for unforeseen setbacks.

Third-party localisation into languages beyond English will be evaluated after the open beta, based on player demographic data and commercial viability.

8.3 Estimated Production Workflow

The GANTT below refers to the development of the prototype, while the release of the final game is estimated to take place in 2028 and will be an extension of the phases described in the GANTT itself, but with a more detailed breakdown of the tasks and a longer timeline. After the demo day, the developers will focus on adding the necessary elements to complete the first area, bug fixing, and testing.

The completion of the first playable area is estimated to be in January 2027.

In particular, the development will focus on:

- ### GANTT CHART GRAPH

Figure 8: Estimated GANTT for the development of the prototype.

9 Cost Estimation and Financial Strategy

The financial planning for Juggernaut adopts a “Lean Startup” approach, optimized for a 24-month development cycle covering pre-production, prototype, and open beta phases. This budget does not cover the commercial release phase; post-beta infrastructure costs (including the migration to a dedicated-server architecture described in §5.5) will be re-estimated on the basis of real usage data collected during the open beta. A core pillar of our market entry strategy is the implementation of a **Friend Pass** system, which allows a single purchasing user to invite a friend to play the full co-op experience for free via Steam.

9.1 Human Resources and Compensation

The project is driven by three core founders (the developers) and supplemented by specialized external talent.

- **Internal Team (Founders):** To ensure long-term sustainability, the three developers will receive a “Founder’s Living Stipend” of €1,500/month. This minimizes “burn rate” while maintaining full-time commitment.
- **External Freelancers:** A budget of €30,000 is allocated for Digital Art and Sound Engineering to ensure the high-quality audiovisual feedback necessary for a competitive shooter.

9.2 Technical Infrastructure and Friend Pass Impact

The Friend Pass system introduces specific technical and financial considerations:

- **Server Costs:** Since one sale can result in two concurrent active users, the cloud backend budget (AWS: EC2 + RDS + ALB, as described in §5.2) has been set at €350/month to account for the higher load-to-revenue ratio and to include a safety margin for traffic spikes during the open beta. This figure covers the beta architecture only; the commercial architecture based on AWS GameLift will require a separate cost analysis.
- **Licensing:** Unity Pro licenses are maintained at €540/month for the core development team.

9.3 Consolidated Budget Table

The following table outlines the 24-month expenditure, including a 10% contingency fund for technical debt or server spikes during the launch phase.

Category	Monthly Cost	Total (24 Months)
Founders' Compensation (3 Devs)	€ 4,500	€ 108,000
Digital Art & Assets (External)	-	€ 20,000
Sound Engineering (External)	-	€ 10,000
Unity Pro Licenses	€ 540	€ 12,960
Cloud Backend (Adjusted for Friend Pass)	€ 350	€ 8,400
Coworking Space Rent	€ 1,200	€ 28,800
Hardware, Trailer & Social Ads	-	€ 17,600
Subtotal		€ 205,760
Contingency Fund (10%)	-	€ 20,576
TOTAL ESTIMATED BUDGET		€ 226,336

Table 15: Consolidated Development and Open Beta Infrastructure Budget (24 months)

9.4 Business Model: Friend Pass and Break-even Analysis

The Friend Pass serves as an organic marketing tool, reducing the **Customer Acquisition Cost (CAC)** by turning every buyer into a brand advocate.

- **Unit Price:** €19.99 (Premium positioning due to the "2-for-1" value proposition).
- **Break-even Point:** Accounting for the 30% Steam fee, the project reaches its break-even point at approximately 16,200 units sold ($€226,336 \div €13.99$ net per unit). This figure represents the development and beta cost recovery; ongoing server costs post-launch will affect the long-term margin but are not included in this estimate.

While the Friend Pass doubles the server load per sale, it significantly increases the "Daily Active Users" (DAU), which is a vital metric for the game's visibility on digital storefronts and long-term community health.